Computer Vision

Tomas Conti

March 11, 2026

Abstract

This project aims to develop a neural network for classifying smartphone-captured images of grocery store products. The work is structured into two distinct tasks: the first involves implementing a custom convolutional neural network for image classification from scratch, while the second focuses on fine-tuning a pre-trained network.

1 Introduction

The Grocery Store Dataset constitutes the foundation of this project, comprising natural images of products captured with smartphone cameras in various grocery store environments. The dataset link is:

`https://github.com/marcusklasson/GroceryStoreDataset.git`

This repository contains 5,125 natural images spanning 81 distinct classes, including fruits, vegetables, and carton-based items such as juice, milk, and yoghurt. All images were acquired under realistic conditions using smartphone cameras in actual grocery stores, resulting in a challenging classification task that reflects real-world scenarios.

The dataset comes pre-partitioned into training, validation, and testing splits. Only the training and validation splits will be manage during model development and evaluation. The test set is a kind of a black box that will be open at the end for the models performance comparison.

The project git hub is :

`https://github.com/TomasConti02/ComputerVisionProject`

2 Dataset examination

Examination of the dataset distribution and label frequency reveals that the validation split does not contain instances from all available classes. This imbalance setting is a deliberate design decision by the dataset creators. The rarest categories are intentionally omitted from the validation set and appear only in the training split.

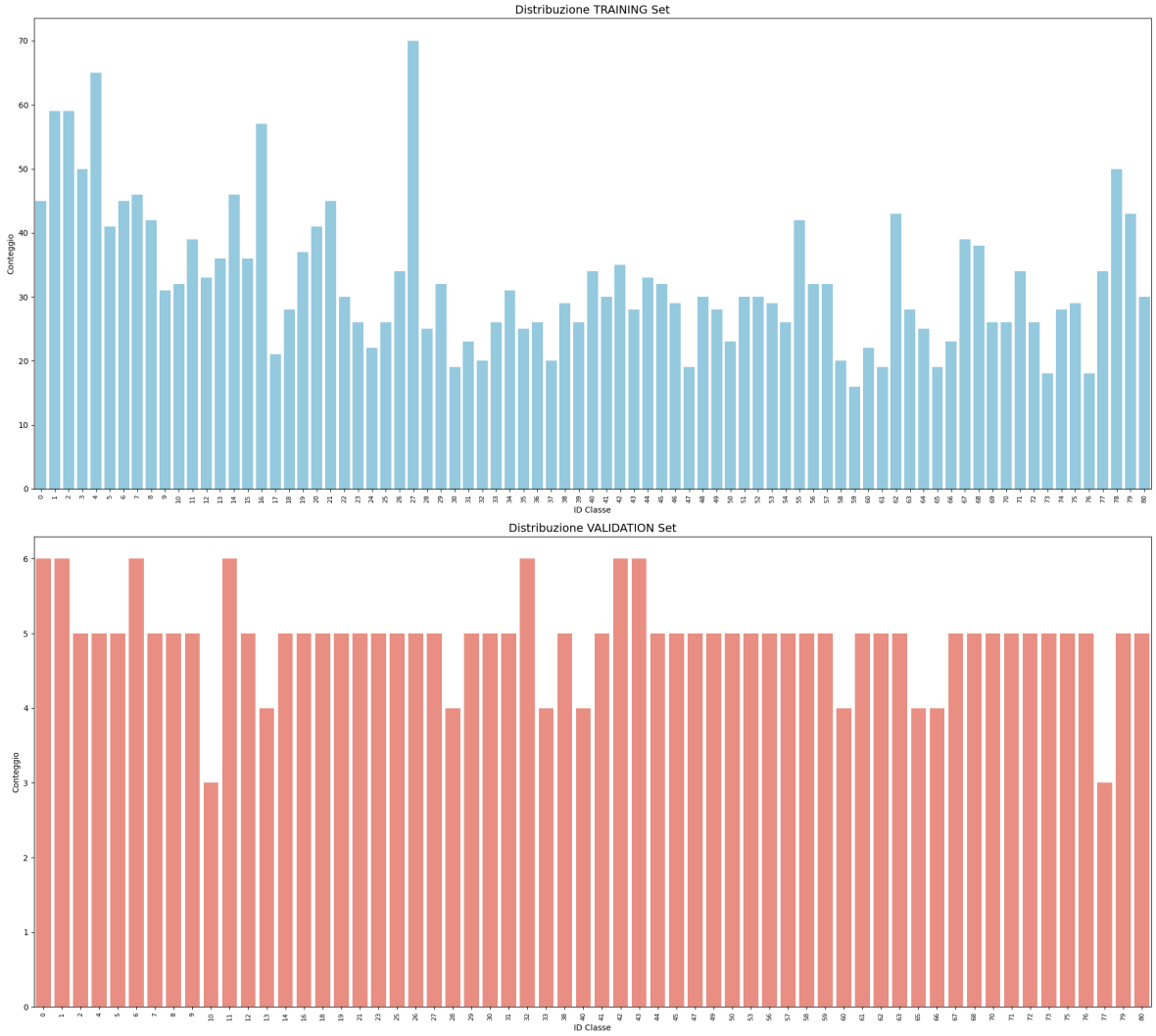

Figure 1: Original dataset labels class distribution

Given the absence of certain classes in the original validation split, I opted to restructure the dataset to ensure a more comprehensive evaluation. Specifically, merge the original training and validation sets into a single unified dataset, then performed a new train-validation split with an 80% and %20 ratio. This approach guarantees that:

- Complete Class Coverage: All 81 classes are represented in both the training and validation splits, enabling a thorough assessment of the model's performance across the entire label space.
- Balanced Evaluation: The validation metrics now reflect the model's capability on every category, including the rarest ones that were previously excluded from evaluation.
- Preserved Data Ratio: By maintaining an 80/20 proportion, we retain a sufficiently large training set while ensuring a statistically meaningful validation set for performance monitoring and keep the original ratio.

This restructuring was implemented using a stratified splitting strategy to preserve class distribution proportions in both the training and validation subsets.

Figure 2: New dataset labels class distribution

Following the dataset restructuring, we observed that the original class distribution remained inherently imbalanced, with certain categories appearing significantly more frequently than others. To mitigate the potential bias toward majority classes during training, we implemented a class weighting strategy.

In imbalanced datasets, standard training operations tend to favor majority classes, as the overall loss is dominated by the numerous samples from these categories. Consequently, the model may achieve high accuracy on frequent classes while performing poorly on rare ones. This behavior is particularly problematic given our objective to evaluate performance across all 81 classes.

This approach ensures that the contribution of each sample to the loss function is inversely proportional to its class frequency. During training, the loss for a sample from class is multiplied by weight, effectively balancing the gradient updates across all categories. What I'm going to do is increase the weight update effect if the model fail in the classification of a rare class (with a few samples).

3 First Task: CNN Architecture from Scratch

The objective of this task is to implement a convolutional neural network for image classification and train it on the Grocery Store Dataset. The target is to achieve a classification accuracy of approximately 60% on the validation split.

Based on the analysis of established VGG architectures and the complexity of the Grocery Store classi-

fication task, I opted to implement a custom architecture inspired by the VGG-11 model characterized by its 11 weighted layers (8 convolutional and 3 fully connected). The task is particularly heavy with a 81 labels to be classified so a deep model is able to detect more interesting and complex feature.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**

Figure 3: VGGs architectures.

I carry out a more specific version of the neural network standard architecture.

Each 3×3 Conv block with a stride of one adopts "same" zero-padding to maintain consistent spatial dimensions for all the feature extraction process (VGG standard). The convolutional output is passed through a ReLU activation function to introduce non-linearity and enhance representational capacity.

A 2×2 Max Pooling layer with a stride of 2 is then applied, without overlapping or padding, resulting in a 50% activation shape reduction rather than input. This downsampling step promotes feature aggregation and accelerates the expansion of the receptive field, enabling the network to capture higher-level contextual information in deeper layers.

The three fully connected layers have been replaced with a single dense layer preceded by Global Average Pooling (GAP), and Batch Normalization was incorporated after each convolutional layer.

- Global Average Pooling (GAP):
  - *Parameter Reduction:* This replacement reduce total parameters, effectively mitigating overfitting on small dataset.
  - *Computational Efficiency:* Increase the training speeds upa and feature aggregation.
- Batch Normalization:
  - *Accelerated Convergence:* Reduces internal covariate shift, allowing higher learning rates and faster training. Standardizes the activations enforcing zero mean and unit variance.
  - *Gradient Flow:* Mitigates exploding and vanishing gradients, enabling effective training of the layers.

The training generator shuffles the data at each epoch to prevent cyclical patterns, while the validation generator maintains a fixed order for consistent evaluation as in the first task model.

Table 1: Custom VGG Model Architecture (Full Version)

Layer (Type)	**Output Shape**	**Parameters**
Input Layer	(256, 256, 3)	0
RandomCrop	(224, 224, 3)	0
RandomFlip	(224, 224, 3)	0
RandomRotation	(224, 224, 3)	0
RandomContrast	(224, 224, 3)	0
RandomTranslation	(224, 224, 3)	0
Conv2D (64 filters) + BatchNorm	(224, 224, 64)	2,048
MaxPooling2D	(112, 112, 64)	0
Conv2D (128 filters) + BatchNorm	(112, 112, 128)	74,368
MaxPooling2D	(56, 56, 128)	0
Conv2D (256 filters) + BatchNorm	(56, 56, 256)	296,192
Conv2D (256 filters) + BatchNorm	(56, 56, 256)	591,104
MaxPooling2D	(28, 28, 256)	0
Conv2D (512 filters) + BatchNorm	(28, 28, 512)	1,182,208
Conv2D (512 filters) + BatchNorm	(28, 28, 512)	2,361,856
MaxPooling2D	(14, 14, 512)	0
Conv2D (512 filters) + BatchNorm	(14, 14, 512)	2,361,856
Conv2D (512 filters) + BatchNorm	(14, 14, 512)	2,361,856
MaxPooling2D	(7, 7, 512)	0
GlobalAveragePooling2D	(512)	0
Dense (512 units)	(512)	262,656
Dropout of 50%	(512)	0
Dense (Output, 81 classes)	(81)	41,553
Total Parameters	9,535,697 ($\approx$ 36.38 MB)	

Input pixel values have been rescaled from $[0, 255]$ to $[0, 1]$ to improve numerical stability during training. Neural networks are sensitive to the scale of input features, and large pixel magnitudes can lead to large activations and gradients, potentially causing unstable updates and slower convergence.

A batch size of 64 was selected to balance training speed and gradient stability, leveraging the memory capacity of the Tesla T4 GPU available in the Google Colab environment.

- Internal Data Augmentation: Integrating augmentation layers directly into the model enables GPU-accelerated preprocessing.
- Label Smoothing: This regularization technique mitigates overconfidence and enhances model calibration by replacing hard one-hot encoded targets with softened label distributions. Instead of forcing the model to assign full probability to the correct class and zero to all others, label smoothing introduces a small uniform distribution across incorrect classes. This prevents the model from becoming overly confident on training samples, reducing overfitting and improving generalization to unseen data. Importantly, it imposes a penalty when the model assigns zero probability to any class, promoting a more balanced probability distribution and greater robustness, especially in settings with many classes or ambiguous inter-class boundaries.
- Class Weights: The balanced class weighting ensures that the model's performance is not biased toward majority classes, leading to more equitable performance across all 81 categories.

Given the limited size of the Grocery Store Dataset, data augmentation is crucial to artificially expand the training set and improve model robustness to real-world variations. The following augmentation pipeline techniques were applied during training. In addiction some of the model hyperparameter for the training operation.

Table 2: Summary of Data augmentation choices.

Data augmentation	**Setting**
RandomCrop	from 256×256 to 224×224
RandomFlip	horizontal
RandomRotation	±20°
RandomContrast	10%
RandomTranslation	10%, −10%

Table 3: Summary of network hyperparameters and design choices.

Hyperparameter	**Value / Setting**
Regularization L2	0.01
Dropout	50%
Learning rate optimizer	0.0004
Batch size	64
Epochs	60

The experiments have been conducted on a Google Colab environment equipped with a Tesla T4 GPU. After systematic hyperparameter tuning and multiple training iterations, I obtained the following results.

The training shows strong initial overfitting, overcome by an effective learning phase between epochs 14-18. After long fluctuations between 41-50, ReduceLROnPlateau reduced the learning rate allowing the model to reach its peak: 98.% validation accuracy, with drop loss.

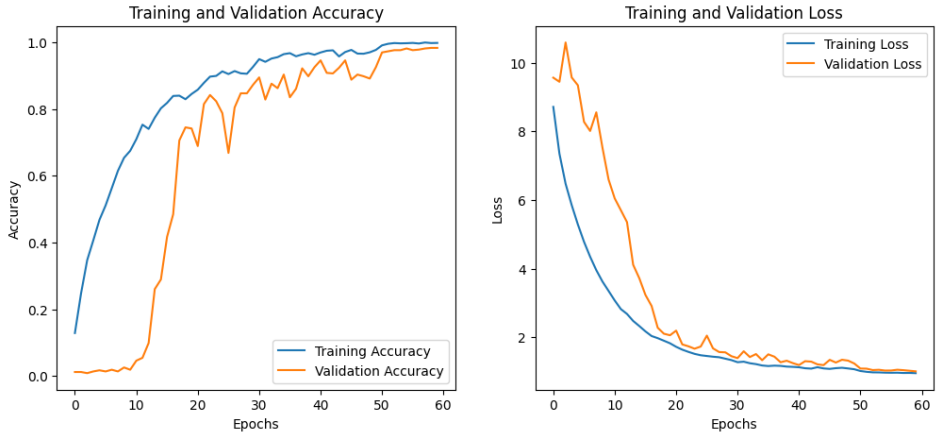

Figure 4: Training and Validation accuracy and loss.

Between epochs 20 and 50, there is a validation accuracy fluctuation because the network has been trained from scratch. With no pre-trained knowledge to rely on, the model quickly memorizes the training data, causing the validation metrics to become unstable and eventually degrade, with no clear stabilization even after 50 epochs.

It is likely that the gradient descent was approaching a local minimum during the 'spike' epoch, but the learning rate was too high to converge. The loss only stabilized after the learning rate was tuned (reduced), allowing the model to settle into the minimum.

4 Second Task: ResNet-18 Transfer Learning

The objective of this task is to fine-tune a pre-trained ResNet-18 model on the Grocery Store Dataset. Unlike the first task, where a custom implementation was required, here we leverage the implementation

provided by Keras through the classifiers module. Specifically, we use the ResNet-18 model pre-trained on ImageNet dataset. Link for pre-train model:

`https://github.com/qubvel/classification_models`

Dataset pre-train model link:

`https://www.image-net.org/`

The data generators are configured with the ResNet-18-specific preprocessing function to ensure all input images are properly managed before entering the network according to the system. The data augmentation pipeline remains unchanged from the previous task. Additionally, the class imbalance weighting and label smoothing strategies have been kept the same. keeping the same architecture to the original design idea allow a good comparison between the two solutions.

The ResNet-18 model architecture has been extended with a custom classification head and integrated data augmentation pieline layers. Following the augmentation layers (invariant from the first task), the pre-trained ResNet-18 backbone is applied without its original classification head. The feature maps produced by the backbone are then passed to a previously designed classification head consisting of:

- A Global Average Pooling layer, which reduces spatial dimensions while preserving channel information
- A dense layer with 512 units and ReLU activation, regularized with L2 regularization (0.001)
- Dropout with a rate of 0.5 to prevent overfitting
- A final dense layer with softmax activation producing predictions across the 81 classes

The fully connected regulation has been reduced from 0.01 to 0.001, in the test phase provides better results. The fine-tuning process follows a warm-up transfer learning approach structured into two distinct phases.

A initial Warm-Up phase, the pre-trained ResNet-18 backbone is frozen, and only the newly added classification head is trained. We initialize the head weights specifically for the Grocery Store Dataset while keeping the feature extractor frozen to preserve its learned representations. A learning rate of 4×10^{-4}(fast learning from scratch) is used, which is appropriate for training new layers from scratch.

A second Fine-Tuning phase, the entire network is unfrozen and trained end-to-end with a reduced learning rate of 1×10^{-4} (slow learning to avoid catastrophic forgetting). The weights learned during the warm-up phase are preserved and transferred to the new model, ensuring that the classification head's progress is not lost.

This gradual unfreezing strategy enables the pre-trained features to be fine-tuned on specific characteristics of grocery store obejcts while preserving the general knowledge acquired from ImageNet dataset. The reduced learning rate ensures that updates to the pre-trained weights are conservative, avoiding catastrophic forgetting while still allowing adaptation to the target domain.

This two-phase strategy is motivated by several considerations:

- Preservation of Pre-trained Features: The warm-up phase prevents destructive updates to the robust feature representations learned from ImageNet, allowing the classification head to stabilize before modifying the backbone.
- Learning Rate Differentiation: The reduced learning rate in the second phase enables local minima convergence by making smaller, more refined updates to the network parameters.
- Computational Efficiency: The warm-up phase requires significantly less computation, allowing rapid prototyping of the classification head before committing to full fine-tuning.

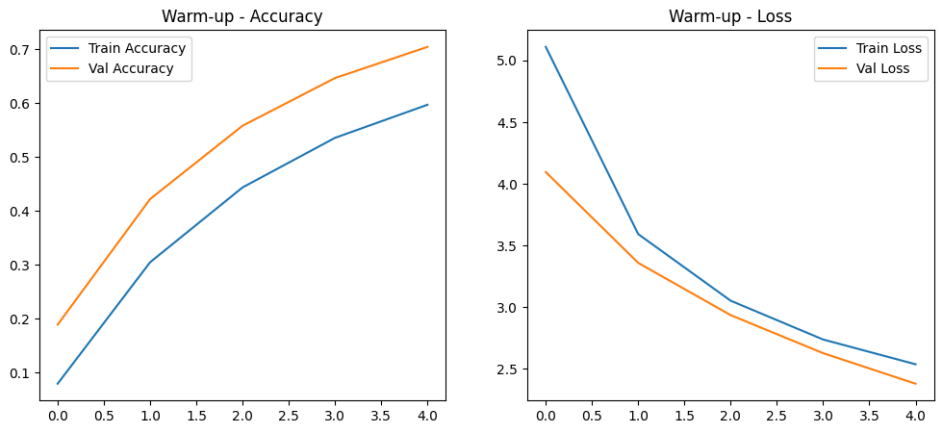

Figure 5: Warm up training and validation accuracy and loss.

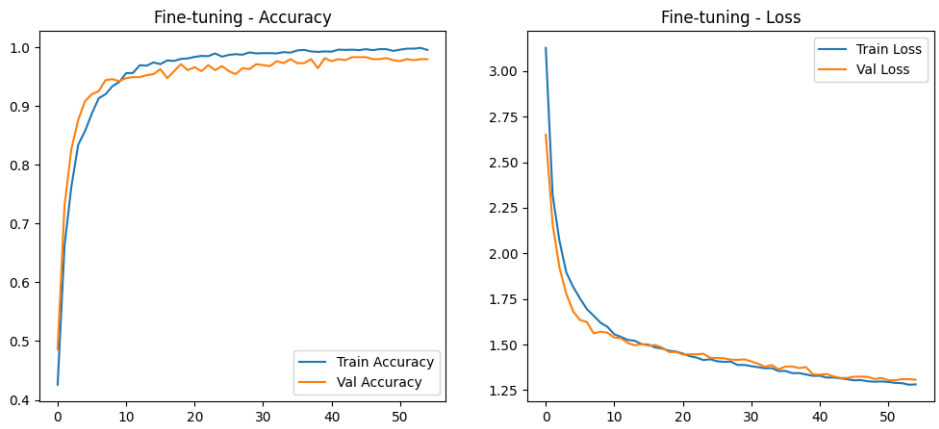

Figure 6: Tuning training and validation accuracy and loss.

The training doesn't show up any catastrophic forgotten and no train fluctuation. The two-phase training strategy successfully achieved the objective, with final validation accuracy reaching 98%, well above the 80-90% target range.

The warm-up phase, training only the classification head with frozen backbone, demonstrated rapid convergence. Fast improving of the validation accuracy and rapid decrease of validation loss to prepare the new weight for the new grocery store domain.

Unfreezing the backbone with a reduced learning rate allows for a good, stable training process. The backbone weights, which carry all the knowledge from the pre-trained dataset, begin to adapt to the new data and quickly surpass the peak reached during the warm-up phase. The process culminated in a high final validation accuracy and a low validation loss. Throughout this phase, training and validation metrics remained closely aligned, confirming that the model maintained strong generalization without signs of overfitting or pattern forgetting.

The results validate the chosen approach: warm-up stabilizes the classification head, fine-tuning adapts the backbone to the target domain, and conservative learning rates with scheduling ensure smooth convergence. The absence of catastrophic forgetting, combined with smooth convergence, demonstrates the effectiveness of the transfer learning strategy for the Grocery Store Dataset.

5 Model Performance Comparison and Conclusions

The best model built from scratch achieved 75% accuracy on the test set. In comparison, transfer learning with fine-tuning reached 87% accuracy. The latter approach proves more effective with small datasets, offering improved generalization and feature learning for the classification task.

The final figure shows the confusion matrix of the second task model's predictions on test set.

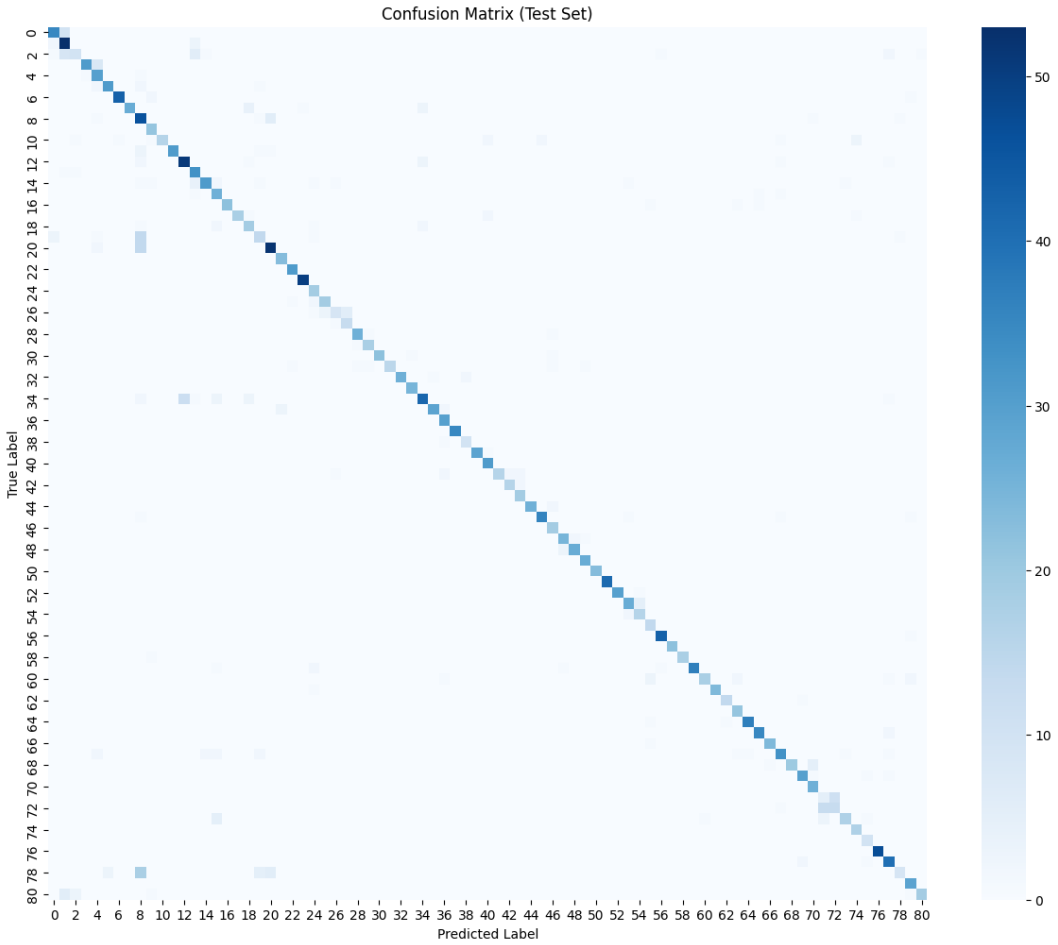

Figure 7: Confusion matrix of the second task model on taset set.